
User:

Give me top 100 interview asked spring boot question set with answer to crack interview with 2+ years of exp. Remember answers should be quick revision ready with deep knowledge pro tip of needed or deep concepts

ChatGPT:

Here's **Part 1** of your **Top 100 Spring Boot Interview Questions and Answers** tailored for **2+ years experience**, optimized for **quick revision** and **deep understanding**. I'll provide them in batches to keep it manageable and let you revise comfortably.

Spring Boot Interview Questions & Answers (1–20)

1. What is Spring Boot?

Ans: Spring Boot is a Spring module that simplifies the setup, development, and deployment of Spring applications with **auto-configuration**, **embedded servers**, and **starter dependencies**.

Pro Tip: Know that Spring Boot is **opinionated**—it preconfigures defaults to get you started quickly.

2. What are the key features of Spring Boot?

Ans:

- Auto Configuration
 - Spring Starters
 - Embedded Servers
 - Actuator
 - Minimal XML Configuration
-

3. What is Spring Boot Starter?

Ans: A starter is a set of dependency descriptors that you can include in your application to get specific features (e.g., `spring-boot-starter-web` includes Tomcat, Jackson, Spring MVC).

4. What is Spring Boot Auto Configuration?

Ans: Automatically configures Spring application based on dependencies in the classpath using `@EnableAutoConfiguration`.

5. What is the role of `@SpringBootApplication`?

Ans: It's a convenience annotation that combines:

- `@Configuration`
 - `@EnableAutoConfiguration`
 - `@ComponentScan`
-

6. How does Spring Boot handle dependency management?

Ans: Uses **Spring Boot Starter POMs** and **parent POM** to manage versions and transitive dependencies automatically.

7. What are some commonly used Spring Boot starters?

Ans:

- `spring-boot-starter-web`
 - `spring-boot-starter-data-jpa`
 - `spring-boot-starter-security`
 - `spring-boot-starter-test`
-

8. How do you create a Spring Boot project?

Ans:

- Using Spring Initializr (<https://start.spring.io>)
 - Using Spring Boot CLI
 - Manually via Maven/Gradle project
-

9. What is the default embedded server in Spring Boot?

Ans: Tomcat. Others: Jetty, Undertow.

10. Can we change the default port in Spring Boot?

Ans: Yes, via `application.properties`:
`server.port=8081`

11. What is `application.properties` or `application.yml` in Spring Boot?

Ans: Configuration files used to define app-level properties like port, DB config, logging level, etc.

12. How to enable scheduling in Spring Boot?

Ans: Add `@EnableScheduling` in config class and annotate methods with `@Scheduled`.

13. How to handle exceptions globally in Spring Boot?

Ans: Use `@ControllerAdvice` with `@ExceptionHandler`.

Pro Tip: You can also use `ResponseEntityExceptionHandler` for REST exceptions.

14. What is Spring Boot Actuator?

Ans: Provides production-ready features like monitoring and metrics via endpoints like `/actuator/health`, `/metrics`, etc.

15. What is the use of `@RestController`?

Ans: Combines `@Controller` + `@ResponseBody` to return JSON/XML directly.

16. Difference between `@Controller` and `@RestController`?

Ans:

- `@Controller`: Returns view (e.g., Thymeleaf, JSP)
 - `@RestController`: Returns response body (JSON/XML)
-

17. How does Spring Boot connect to a database?

Ans: Define DB properties in `application.properties`:

```
properties
spring.datasource.url=...
spring.datasource.username=...
```

It auto-configures `DataSource` and `JPA`.

18. What is Spring Data JPA?

Ans: Simplifies JPA-based database access by extending `JpaRepository` or `CrudRepository`.

19. How to create custom queries in Spring Data JPA?

Ans:

- Using method names: `findByName(String name)`
 - Using `@Query`: `@Query("SELECT u FROM User u WHERE u.name = :name")`
-

20. What is the role of @Entity, @Id, and @GeneratedValue?

Ans:

- @Entity: Marks class as a DB entity
- @Id: Primary key
- @GeneratedValue: Auto-generates ID values

Would you like the next 20 (21–40) now, or shall I format it as a downloadable PDF/Notion style doc too?

User:

Want now in chat all questions with answers please be detailed and quick revision ready but detailed

ChatGPT:

Here's the **complete set of 100 Spring Boot interview questions and answers**, designed for **2+ years experience**, **detailed yet quick to revise**, and covering **deep concepts** with **pro tips** where needed.

Spring Boot Interview Questions & Answers (1–20)

Already shared above — continuing from Question 21.

21. What is CommandLineRunner in Spring Boot?

Ans: It is a functional interface used to run code at startup. Implement its `run()` method.

Use case: DB seeding, data verification at startup.

```
java
@Component
public class MyRunner implements CommandLineRunner {
    public void run(String... args) {
        // code to run at app startup
    }
}
```

22. What is ApplicationRunner?

Ans: Similar to CommandLineRunner, but provides ApplicationArguments.

```
java
```

```
@Component
public class MyAppRunner implements ApplicationRunner {
    public void run(ApplicationArguments args) {
        // access args.getOptionNames()
    }
}
```

23. Difference between CommandLineRunner and ApplicationRunner?

Ans:

- CommandLineRunner: Raw args (String array)
- ApplicationRunner: Structured args (ApplicationArguments)

24. What is @SpringBootTest?

Ans: Used to create an application context for integration testing.

25. What is @DataJpaTest?

Ans: Provides slice test for JPA repository, auto-configures H2 DB, no web environment.

26. How to run Spring Boot app as a JAR?

Ans:

- Package with Maven: `mvn clean install`
- Run with: `java -jar target/app.jar`

27. How to disable a specific auto-configuration?

Ans:

Use `@SpringBootApplication(exclude = DataSourceAutoConfiguration.class)`

28. What is the use of profiles in Spring Boot?

Ans: Allow environment-specific configurations.

`application-dev.properties`, `application-prod.properties`

29. How to set active profile?

Ans:

- CLI: --spring.profiles.active=dev
 - Properties: spring.profiles.active=dev
-

30. What is the default scope of Spring Beans in Spring Boot?

Ans: Singleton. Other scopes: prototype, request, session, application.

31. What is the use of @Value annotation?

Ans: Injects property values into fields.

```
java
@Value("${server.port}")
private int port;
```

32. What is @ConfigurationProperties?

Ans: Binds whole property classes. Used for structured config.

```
java
@ConfigurationProperties(prefix="app")
public class AppConfig {
    private String name;
}
```

33. What is @EnableConfigurationProperties?

Ans: Enables binding of external properties to POJO classes marked with @ConfigurationProperties.

34. What is the difference between @Value and @ConfigurationProperties?

Ans:

- @Value: One-off injection
 - @ConfigurationProperties: Bulk mapping into bean, good for complex configs
-

35. What is the purpose of @EnableAutoConfiguration?

Ans: Triggers auto-configuration of Spring beans based on classpath.

36. What is the order of precedence for Spring Boot configurations?

Ans:

1. Command-line args
 2. application.properties/yml
 3. OS env vars
 4. Default values in code
-

37. What is a bean?

Ans: A Spring-managed object created, configured, and managed in the Spring container.

38. What is the difference between @Component, @Service, @Repository, and @Controller?

Ans:

- @Component: Generic bean
 - @Service: Business logic
 - @Repository: DAO, enables exception translation
 - @Controller: MVC controller
-

39. How to consume REST API using RestTemplate?

Ans:

```
java
RestTemplate restTemplate = new RestTemplate();
String result = restTemplate.getForObject(url, String.class);
```

40. What is WebClient in Spring Boot?

Ans: Non-blocking reactive alternative to RestTemplate. Used in Spring WebFlux.

Spring Boot Questions (41–60)

41. What is Spring WebFlux?

Ans: A reactive-stack web framework to handle async and non-blocking requests using Project Reactor.

42. What is @Bean annotation?

Ans: Marks a method that returns a bean to be managed by Spring container.

43. What is DevTools in Spring Boot?

Ans: Enables auto-restart, live reload, and dev-time configuration for faster development.

44. How to handle cross-origin requests (CORS)?

Ans: Use @CrossOrigin on controller/method or define global CORS config.

45. How to configure custom error pages?

Ans: Create /error mapping or place error.html in resources/templates.

46. What is Spring Security?

Ans: A framework for securing Spring apps with features like authentication, authorization, CSRF, etc.

47. How to secure REST endpoints in Spring Boot?

Ans:

- Add spring-boot-starter-security
 - Extend WebSecurityConfigurerAdapter (deprecated, use SecurityFilterChain now)
-

48. How to implement JWT with Spring Boot?

Ans:

- Intercept requests with filters
 - Validate JWT token
 - Set authentication in SecurityContext
-

49. How to enable HTTPS in Spring Boot?

Ans: Add keystore properties in `application.properties` and configure SSL.

50. What is actuator endpoint `/health` used for?

Ans: Exposes application health status (e.g., DB connection, disk space)

51. How to monitor metrics in Spring Boot?

Ans: Use Actuator + Micrometer. Integrate with Prometheus, Grafana.

52. What is `@Transactional`?

Ans: Declares that method/class should be run within a transaction.

53. What is lazy initialization in Spring Boot?

Ans: Beans are created only when needed. Enable with `spring.main.lazy-initialization=true`.

54. What is Spring Boot CLI?

Ans: Command-line interface for running Groovy scripts to build Spring apps.

55. What is dependency injection?

Ans: Design pattern where dependencies are provided externally (constructor/setter-based).

56. Constructor vs Setter injection?

Ans:

- Constructor: Mandatory dependencies
 - Setter: Optional dependencies
-

57. How to run code only once after app starts?

Ans: Implement `ApplicationRunner` or `CommandLineRunner`.

58. How to override default properties in tests?

Ans: Use `@TestPropertySource` or define in `application-test.properties`.

59. How to exclude auto-config classes?

Ans:

```
java
@SpringBootApplication(exclude = { DataSourceAutoConfiguration.class })
```

60. What is `@ConditionalOnMissingBean`?

Ans: Configures a bean only if one isn't already present in the context.

Let me know if you'd like me to continue with **Questions 61–100** right away!

User:

Yes

ChatGPT:

Here's the **final batch: Spring Boot Interview Questions & Answers (61–100)** — optimized for **2+ years of experience**, quick revision, and deep understanding.

Spring Boot Questions (61–80)

****61. What is the role of Spring Boot Parent in `pom.xml`?**

Ans: It manages default versions of dependencies, plugins, and configurations — simplifies Maven config.

```
xml
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.x.x</version>
</parent>
```

62. How to externalize configuration in Spring Boot?

Ans:

- application.properties or .yaml
 - Command line arguments
 - OS environment variables
 - Config Server (for microservices)
-

63. How to load a custom configuration file?

Ans:

```
bash
--spring.config.location=classpath:/myconfig.properties
```

64. How to log SQL queries in Spring Boot with JPA?

Ans:

```
properties
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
logging.level.org.hibernate.SQL=DEBUG
```

65. How to enable H2 console in Spring Boot?

Ans:

```
properties
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
```

66. What is @EntityScan used for?

Ans: Tells Spring where to scan for entity classes if not in main package.

67. What is the difference between CrudRepository, JpaRepository, and PagingAndSortingRepository?

Ans:

- CrudRepository: Basic CRUD

- JpaRepository: Adds batch, flush, etc.
 - PagingAndSortingRepository: Adds pagination/sorting
-

68. What is OpenFeign in Spring Boot?

Ans: Declarative REST client for calling other REST services using interface and annotations.

```
java
@FeignClient(name = "user-service")
public interface UserClient {
    @GetMapping("/users/{id}")
    User getUser(@PathVariable Long id);
}
```

69. What is circuit breaker in microservices?

Ans: Prevents cascading failures; fallback mechanism for failed services (e.g., using Resilience4j, Hystrix).

70. What is Spring Cloud Config?

Ans: Centralized external configuration for distributed systems.

71. How to make Spring Boot app stateless?

Ans:

- Don't use session
 - Use JWT tokens for authentication
 - Set stateless() in Spring Security config
-

72. How to implement pagination with Spring Data JPA?

Ans: Use Pageable in repository methods.

```
java
Page<User> findByActiveTrue(Pageable pageable);
```

73. How to log request and response in Spring Boot REST?

Ans:

- Use filters/interceptors
 - Enable actuator httptrace
 - Add AOP or use logging frameworks
-

74. Difference between @RequestParam, @PathVariable, @RequestBody?

Ans:

- @RequestParam: From query params
 - @PathVariable: From URI
 - @RequestBody: From JSON body
-

75. What is spring-boot-devtools used for?

Ans: Speeds up development with auto-restart, live reload, and better logging.

76. What is spring.main.allow-bean-definition-overriding=true?

Ans: Allows multiple beans of same name; last one overrides others.

77. What is caching in Spring Boot?

Ans:

- Use @EnableCaching, @Cacheable, @CacheEvict
 - Works with Redis, Ehcache, Caffeine, etc.
-

78. How to enable async execution?

Ans:

- Add @EnableAsync
 - Use @Async on method
 - Return CompletableFuture or Future
-

79. What is @Scheduled in Spring Boot?

Ans: Used for scheduled tasks.

```
java
@Scheduled(fixedRate = 5000)
public void runJob() { ... }
```

80. What is actuator endpoint /env used for?

Ans: Shows environment properties, active profiles, etc. (Secure it in prod!)

Spring Boot Questions (81–100)

81. What is the difference between Mono and Flux in WebFlux?

Ans:

- Mono: 0–1 elements
 - Flux: 0–N elements (streams)
-

82. How to validate request data in Spring Boot?

Ans:

- Use @Valid with @RequestBody
 - Annotate model with JSR-303 annotations (@NotNull, @Size, etc.)
-

83. How to gracefully shut down Spring Boot app?

Ans:

- Enable in application.properties:

```
properties
server.shutdown=graceful
```

- Implement DisposableBean or @PreDestroy
-

84. What is reactive programming?

Ans: Async, non-blocking programming model using backpressure and event streams.

85. How to expose only selected actuator endpoints?

Ans:

```
properties
management.endpoints.web.exposure.include=health,info
```

86. How to auto-generate schema with JPA?

Ans:

```
properties
spring.jpa.hibernate.ddl-auto=create|update|validate|none
```

87. How to ignore null fields in JSON response?

Ans:

```
java
@JsonInclude(JsonInclude.Include.NON_NULL)
```

88. What is Flyway and Liquibase?

Ans: DB migration tools to version, track, and apply schema changes.

89. What is actuator /beans endpoint for?

Ans: Lists all registered beans in the application context.

90. How to encrypt properties in Spring Boot?

Ans: Use jasypt-spring-boot or custom PropertySource.

91. How to reload properties at runtime?

Ans: Use Spring Cloud Config + Actuator /refresh, or manually reload the bean using @RefreshScope.

92. What is @Profile annotation?

Ans: Allows beans to be loaded only in specific environment profiles.

93. How to enable hot reloading in IntelliJ or Eclipse?

Ans:

- Add DevTools
 - Enable auto-build in IDE
 - Restart trigger on classpath change
-

94. What is @RequestScope, @SessionScope, @ApplicationScope?

Ans: Bean scopes:

- Request: Per HTTP request
 - Session: Per HTTP session
 - Application: Singleton for servlet context
-

95. What is role of @ComponentScan?

Ans: Scans specified package for Spring components.

96. What is Spring Boot Admin?

Ans: Community project to monitor and manage Spring Boot apps via a UI.

97. What is the difference between microservices and monolith in Spring Boot?

Ans:

- **Monolith:** Single deployable unit
 - **Microservice:** Independent, loosely coupled services communicating over HTTP/Message queues
-

98. How to run Spring Boot app inside Docker?

Ans:

- Create Dockerfile

```
dockerfile
```

```
FROM openjdk:17
COPY target/app.jar app.jar
ENTRYPOINT ["java","-jar","/app.jar"]
```

99. What are disadvantages of Spring Boot?

Ans:

- Heavy startup time
- Large fat JARs
- Limited control over hidden configurations

100. What are new features in Spring Boot 3.x?

Ans:

- JDK 17+ support
- Native image (GraalVM)
- Jakarta EE 9+
- Improved observability with Micrometer

Let me know if you want:

- A **PDF copy** of this set
 - A **flashcard version**
 - A **Notion-friendly version**
 - Or mock interview questions based on this set!
-